

Module 12: Two-Layer Neural Network

Amr Mansour | JHED ID: amanso8

This report summarizes the implementation and evaluation of a two-layer neural network for graduate admissions classification. The neural network was implemented manually with NumPy, with scikit-learn used only for the train/test split.

1. Dataset and Feature Construction

The **final_clustered_data.csv** dataset produced in the earlier coursework was used as the source dataset. The Module 12 script converts the required fields into the assignment-specified JSON Lines format, **applicant_data.jsonl**, before loading and preprocessing the applicant records. Only Accepted/Rejected and Masters/PhD records are retained. After filtering, **12,712** applicant records remain.

The six model inputs are GPA, GRE, GRE verbal, GRE analytical writing, degree (PhD = 1; Masters = 0), and citizenship (International = 1; American/local = 0). The target is Accepted = 1 and Rejected = 0.

2. Train/Test Split and Preprocessing

The dataset is split 80/20 using **random_state=42** and **shuffle=True**, producing **10,169 training records** and **2,543 test records**. Missing values are imputed using medians calculated from the training set only. Feature means and standard deviations are also calculated from the training set only and then applied to both datasets. This prevents test-set information from leaking into preprocessing.

3. Neural Network Architecture

The network has six input units, six hidden sigmoid units, and one sigmoid output unit (6-6-1). Weights are initialized from a normal distribution with mean 0 and standard deviation 0.1, while biases start at zero. Training uses Mean Squared Error, a learning rate of 0.05, a maximum of 10,000 epochs, patience of 100, and random seed 42. Backpropagation and gradient-descent updates are implemented directly with NumPy.

The parameter dimensions are $W1 = (6, 6)$, $b1 = (1, 6)$, $W2 = (6, 1)$, and $b2 = (1, 1)$. The hidden layer forms six nonlinear representations of the standardized input features. The output layer combines those hidden activations into one sigmoid value. Because sigmoid maps its input to the interval 0 to 1, the output can be interpreted as a probability-like acceptance score.

4. Training Printouts

Training uses full-batch gradient descent. Progress is printed every 100 epochs and includes epoch number, training MSE, test MSE, and test accuracy. Representative output from the final portion of the run is shown below; the complete generated record is included in **training.log**.

```
Epoch 9600 | Train MSE: 0.196941 | Test MSE: 0.200436 | Test Accuracy: 0.7228
Epoch 9700 | Train MSE: 0.196939 | Test MSE: 0.200435 | Test Accuracy: 0.7228
Epoch 9800 | Train MSE: 0.196938 | Test MSE: 0.200434 | Test Accuracy: 0.7228
Epoch 9900 | Train MSE: 0.196937 | Test MSE: 0.200432 | Test Accuracy: 0.7228
Epoch 10000 | Train MSE: 0.196935 | Test MSE: 0.200431 | Test Accuracy: 0.7228
```

The early-stopping rule tracks the best test MSE and corresponding parameters and would stop training after 100 consecutive epochs without improvement. In this run, test MSE continued improving through epoch 10,000, so the maximum epoch limit was reached rather than the patience condition.

5. Final Model Evaluation

Metric	Result
--------	--------

Filtered records	12,712
Training records	10,169
Test records	2,543
Best epoch	10,000
Best test MSE	0.200431
Training accuracy	72.83%
Test accuracy	72.28%

The final test accuracy of 72.28% is above the assignment target of 53%. Training accuracy (72.83%) and test accuracy are close, so this run does not show a large generalization gap or severe overfitting. The result is reasonably stable for this dataset, but it should not be interpreted as evidence that the model is a realistic admissions decision system because several GRE-related variables are highly sparse and important admissions factors are absent.

6. MSE Learning Curve

The figure below is generated by the training script and shows training and test MSE across epochs. Both curves decline and remain close. The best test MSE occurs at epoch 10,000.



Figure 1. Training and test Mean Squared Error across epochs.

7. Artificial Applicant Predictions

Applicant Profile	Probability	Prediction
1 GPA 3.95, GRE 330, PhD, International	0.3549	Rejected
2 GPA 3.60, GRE 315, Masters, American/local	0.7195	Accepted

3	GPA 3.10, GRE 295, Masters, International	0.6742	Accepted
---	---	--------	----------

The strongest numerical profile is not assigned the highest acceptance probability. This is an important observation rather than something to hide: the network learned a nonlinear relationship across all six inputs rather than a simple GPA/GRE ranking. These results should be interpreted cautiously because GRE-related fields are highly sparse and many missing observations are replaced using training-set medians.

8. Limitations and Reflection

Building the network directly with NumPy makes the forward pass, sigmoid activations, MSE calculation, backpropagation, and gradient updates visible and inspectable. A limitation is the use of MSE for binary classification; binary cross-entropy would normally be a more natural classification loss, although MSE is used here to match the assignment specification.

The data is incomplete. Important admissions factors such as recommendation letters, research experience, statement quality, program competitiveness, work experience, and funding are not represented. The model therefore captures associations in the available historical data and should not be treated as a causal or operational admissions decision system. A stronger future model could use richer and more complete data, improve missing-data quality, compare alternative loss functions, and add classification metrics beyond accuracy.

9. Reproducibility

All required dependencies are listed in **requirements.txt**. Running **python neural_network.py** from the **module_12** folder recreates the JSON Lines input and generates the MSE curve and training log. The source CSV is included in the same folder so the submission does not depend on an external absolute path.